

4.12 Structured Programming Summary

Just as architects design buildings by employing the collective wisdom of their profession, so should programmers design programs. Our field is younger than architecture is, and our collective wisdom is considerably sparser. We've learned a great deal in a mere eight decades. Perhaps most important, we've learned that structured programming produces programs that are easier (than unstructured programs) to understand and therefore are easier to test, debug, modify, and even prove correct in a mathematical sense.

Chapters 3 and 4 have concentrated on C's control statements. Each statement has been presented, flowcharted and discussed separately with examples. Now, we summarize the results of Chapters 3 and 4 and introduce a simple set of rules for the formation and properties of structured programs.

Figure 4.17 summarizes the control statements discussed in Chapters 3 and 4. Small circles are used in the figure to indicate the *single entry point* and the *single exit point* of each statement. Connecting individual flowchart symbols arbitrarily can lead to unstructured programs. Therefore, the programming profession has chosen to combine flowchart symbols to form a limited set of control statements, and to build only properly structured programs by combining control statements in two simple ways. For simplicity, only *single-entry/single-exit* control statements are used -there's only one way to enter and only one way to exit each control statement. Connecting control statements in sequence to form structured programs is simple -the exit point of one control statement is connected to the entry point of the next- i.e., the control statements are simply placed one after another in a program -we've called this "control-statement stacking". Control statements also can be nested.

Figure 4.18 shows the rules for forming structured programs. The rules assume that the rectangle flowchart symbol may be used to indicate any action including input/output. Figure 4.19 shows the *simplest flowchart*.

Applying the rules of Fig. 4.18 always results in a structured flowchart with a neat, building-block appearance. Repeatedly applying Rule 2 to the simplest flowchart (Fig. 4.19) results in a structured flowchart containing many rectangles *in sequence* (Fig. 4.20). Rule 2 generates a stack of control statements; so we call Rule 2 the **stacking rule**.

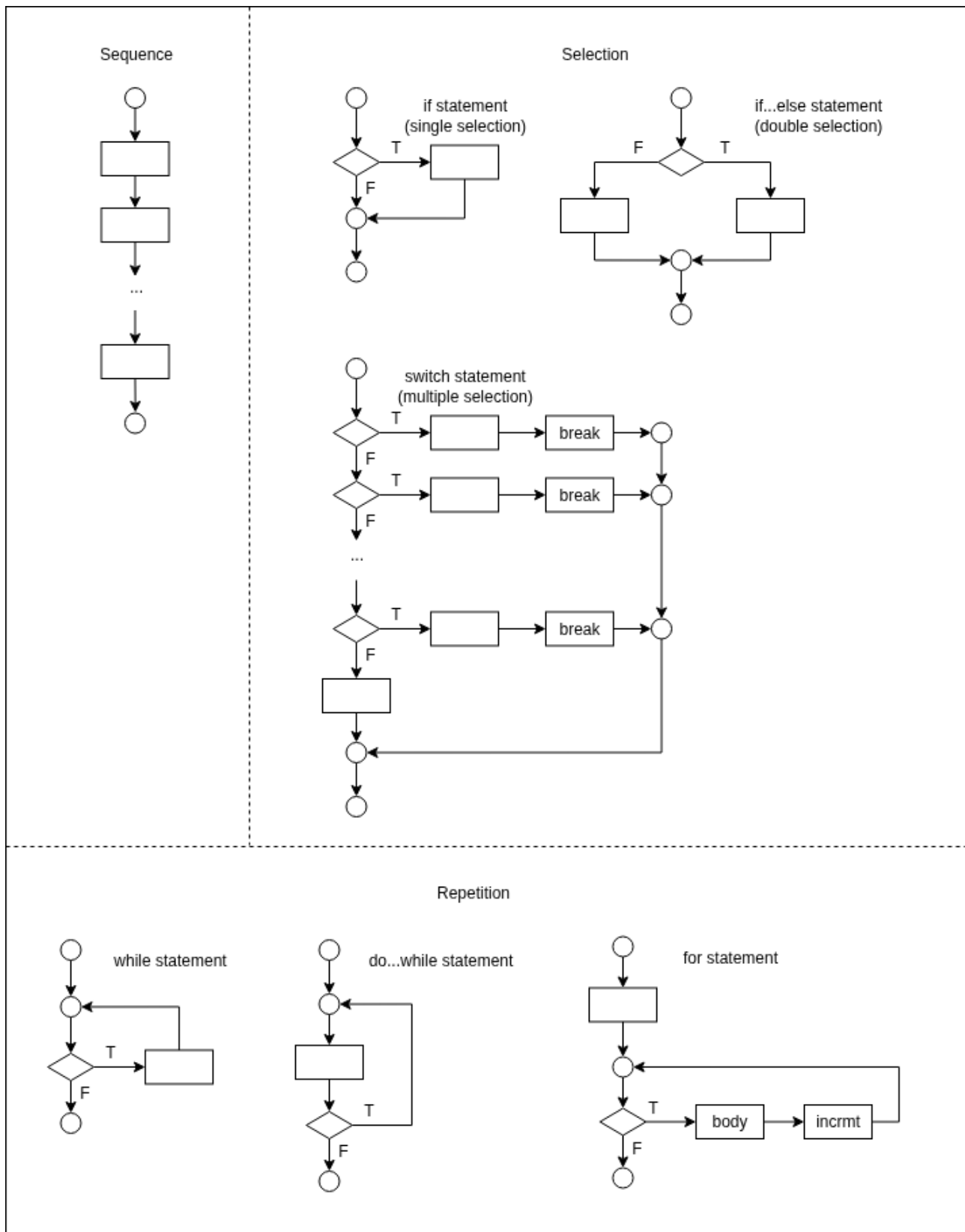


Fig. 4.17 | C's single-entry/single-exit sequence, selection and iteration statements.

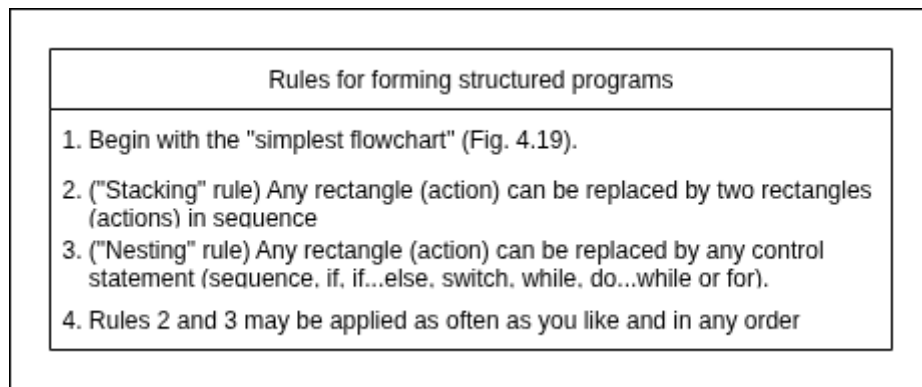


Fig. 4.18 | Rules for forming structured programs.

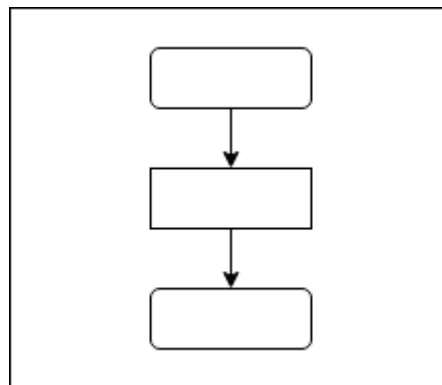


Fig. 4.19 | Simplest flowchart.

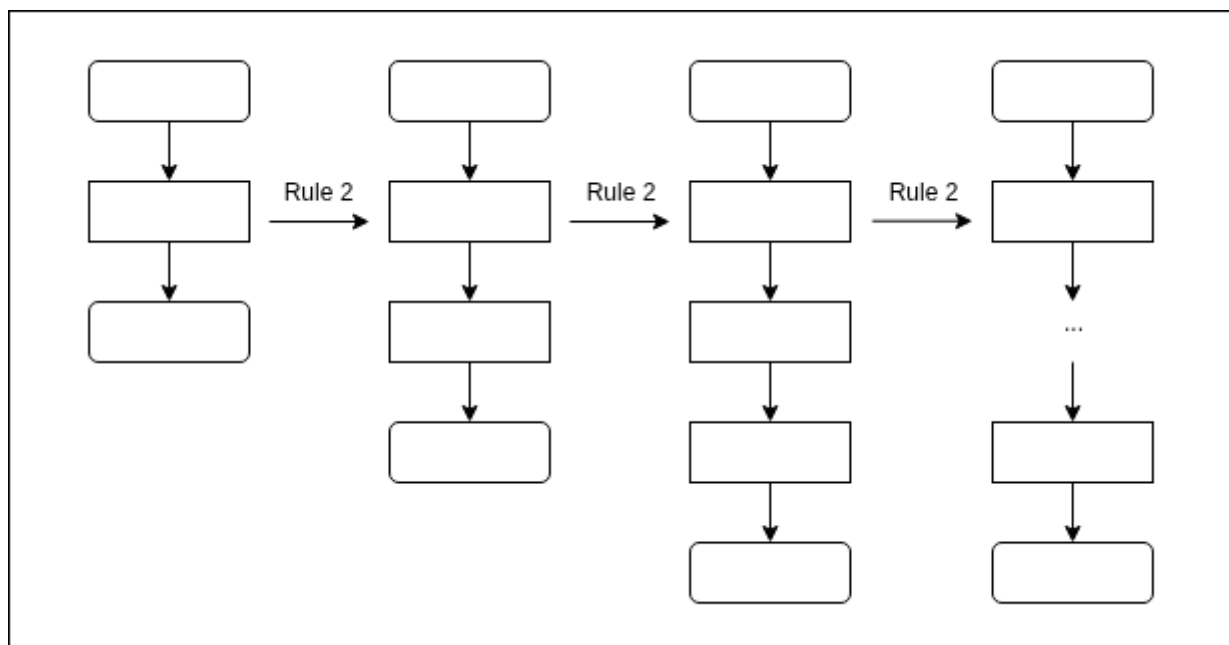


Fig. 4.20 | Repeatedly applying Rule 2 of Fig. 4.18 to the simplest flowchart.

Rule 3 is called the **nesting rule**. Repeatedly applying Rule 3 to the simplest flowchart results in a flowchart with neatly nested control statements. For example, in Fig. 4.21, the rectangle in the simplest flowchart is first replaced with a double-selection (if...else) statement. Then Rule 3 is applied again to both of the rectangles in the double-selection statement, replacing each of these rectangles with double-selection statements. The dashed box around each of the double-selection statements represents the rectangle that was replaced in the original flowchart.

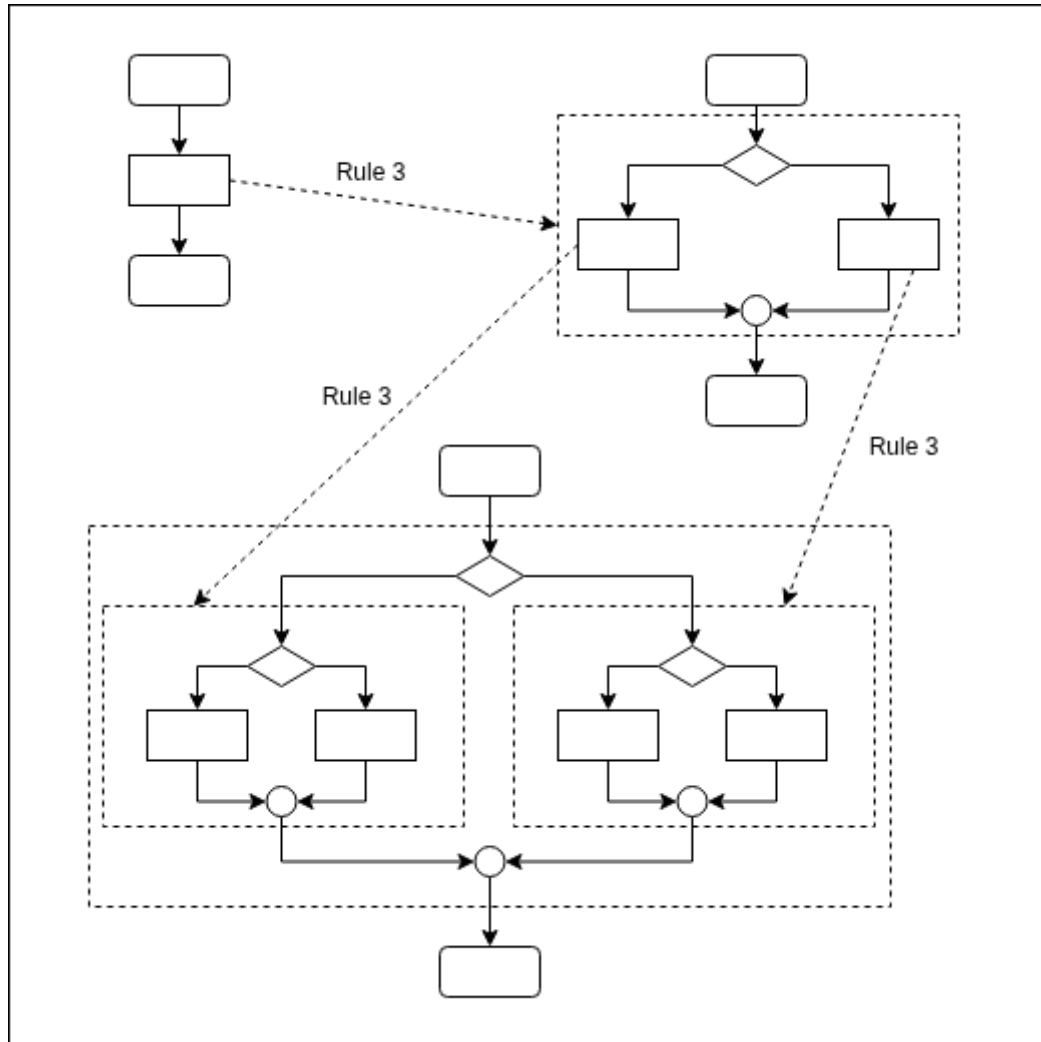


Fig. 4.21 | Applying Rule 3 of Fig. 4.18 to the simplest flowchart.

Rule 4 generates larger, more involved, and more deeply nested structures. The flowcharts that emerge from applying the rules in Fig. 4.18 constitute the set of all possible structured flowcharts and hence the set of all possible structured programs.

It's because of the elimination of the **goto** statement that these building blocks never overlap one another. The beauty of the structured approach is that we use only a small number of simple *single-entry/single-exit* pieces, and we assemble them in only *two* simple ways. Figure 4.22 shows the kinds of stacked building blocks that emerge from applying Rule 2 and the kinds of nested building blocks that emerge from applying Rule 3. The figure also shows the kind of overlapped building blocks that *cannot* appear in structured flowcharts (because of the elimination of the **goto** statement).

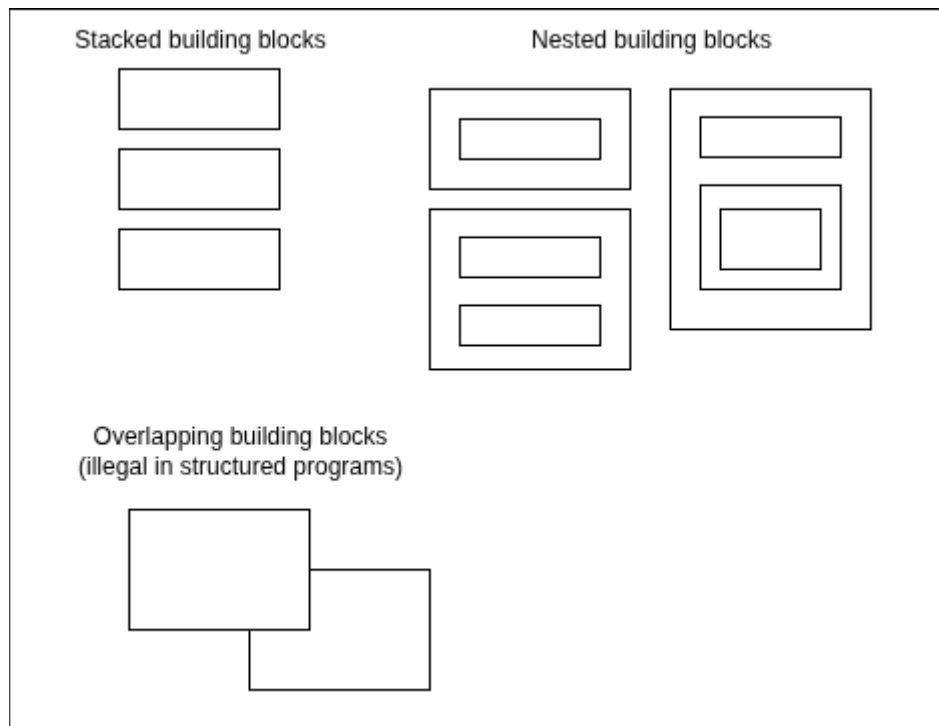


Fig. 4.22 | Stacked, nested and overlapped building blocks.

If the rules in Fig. 4.18 are followed, an unstructured flowchart (such as that in Fig. 4.23) *cannot* be created. If you're uncertain whether a particular flowchart is structured, apply the rules of Fig. 4.18 in reverse to try to *reduce* the flowchart to the simplest flowchart. If you succeed, the original flowchart is structured; otherwise, it's not.

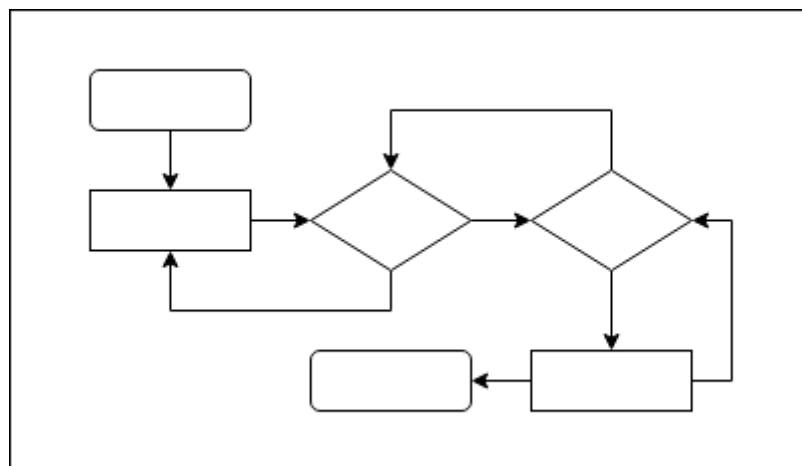


Fig. 4.23 | An unstructured flowchart.

Structured programming promotes simplicity. Bohm and Jacopini showed that only three forms of control are needed:

- Sequence
- Selection
- Iteration

Sequence is straightforward. Selection is implemented in one of three ways:

- if statement (single selection)
- if...else statement (double selection)
- switch statement (multiple selection)

It's straightforward to prove that the simple if statement is sufficient to provide *any* form of selection -everything that can be done with the if...else statement and the switch statement can be implemented with one or more if statements.

It's also straightforward to prove that the while statement is sufficient to provide *any* form of iteration. Everything that can be done with the do...while statement and the for statement can be done with the while statement.

Combining these results illustrates that any form of control ever needed in a C program can be expressed in terms of only *three* forms of control:

- sequence
- if statement (selection)
- while statement (iteration)

And these control statements can be combined in only *two* ways -*stacking* and *nesting*. Indeed, structured programming promotes simplicity.